Chapter 8. Designing concurrent code

This chapter covers

- Techniques for dividing data between threads
- Factors that affect the performance of concurrent code
- How performance factors affect the design of data structures
- Exception safety in multithreaded code
- Scalability
- Example implementations of several parallel algorithms

Most of the preceding chapters have focused on the tools you have in your new C++11 toolbox for writing concurrent code. In chapters 6 and 7 we looked at how to use those tools to design basic data structures that are safe for concurrent access by multiple threads. Much as a carpenter needs to know more than just how to build a hinge or a joint in order to make a cupboard or a table, there's more to designing concurrent code than the design and use of basic data structures. You now need to look at the wider context so you can build bigger structures that perform useful work. I'll be using multithreaded implementations of some of the C++ Standard Library algorithms as examples, but the same principles apply at all scales of an application.

Just as with any programming project, it's vital to think carefully about the design of concurrent code. However, with multithreaded code, there are even more factors to consider than with sequential code. Not only must you think about the usual factors such as encapsulation, coupling, and cohesion (which are amply described in the many books on software design), but you also need to consider which data to share, how to synchronize accesses to that data, which threads need to wait for which other threads to complete certain operations, and so forth.